

ML_HW6_111652040_吳泓諺

1. Code with detailed explanations

i. Part1

```
def compute_gram_matrix(colors, coords, gamma_c, gamma_s):  
    print("Computing Spatial Kernel (Numpy)...")  
    dist_s = compute_sq_dist_mat(coords, coords)  
    K_s = np.exp(-gamma_s * dist_s)  
  
    print("Computing Color Kernel (Numpy)...")  
    dist_c = compute_sq_dist_mat(colors, colors)  
    K_c = np.exp(-gamma_c * dist_c)  
  
    K = K_s * K_c  
    return K
```

Distance Matrices: The `compute_sq_dist_mat` helper function calculates the squared Euclidean distance D between all pairs of data points (pixels).

$$D_{ij} = \|x_i - x_j\|^2$$

Gaussian Kernel (RBF): The function uses the Gaussian (Radial Basis Function) kernel to transform distance into similarity:

$$K(i, j) = \exp(-\gamma \cdot D_{ij})$$

Spatial Kernel (K_s): Measures how close pixels are in the image, controlled by the parameter γ_s . Color Kernel (K_c): Measures how similar the color values are, controlled by the parameter γ_c . Combined Kernel (K): The final kernel K is the element-wise product of K_s and K_c . This ensures that two pixels are considered highly similar only if they are both close in location AND similar in color.

$$K = K_s \odot K_c$$

kernel k-means clustering

```
def kernel_k_means(K, k, h, w, max_iters=100, output_dir='kkm_frames', init_method='random'):  
    if not os.path.exists(output_dir):  
        os.makedirs(output_dir)  
  
    n = K.shape[0]  
  
    # --- Modified: Use initialize_centers for Kernel ---  
    print(f"Initializing Kernel K-means with {init_method}...")  
    labels = initialize_centers(data=None, k=k, method=init_method, for_kernel=True, K_matrix=K)  
  
    K_diag = np.diag(K)  
  
    for it in range(max_iters):  
        print(f"Kernel K-means Iteration {it+1}/{max_iters}")  
        visualize_clusters(labels, h, w, f"{output_dir}/step_{it:03d}.png")  
  
        dist_matrix = np.zeros((n, k))  
  
        for c in range(k):  
            mask = (labels == c)  
            if not np.any(mask):  
                continue  
  
            N_c = np.sum(mask)  
            term2 = -2 * np.sum(K[:, mask], axis=1) / N_c  
            term3 = np.sum(K[np.ix_(mask, mask)]) / (N_c ** 2)  
            dist_matrix[:, c] = K_diag + term2 + term3  
  
        new_labels = np.argmin(dist_matrix, axis=1)  
  
        if np.all(new_labels == labels):  
            print("Converged!")  
            visualize_clusters(labels, h, w, f"{output_dir}/converged.png")  
            break  
        labels = new_labels  
  
    return labels
```

Kernel k -means aims to minimize the within-cluster variance, similar to standard k -means, but uses the kernel trick. The squared distance D^2 between a data point x_i (in the feature space $\phi(x_i)$) and a cluster center μ_c is calculated entirely using the kernel matrix K :

$$\text{Minimize } D^2(x_i, \mu_c) = \|\phi(x_i) - \mu_c\|^2$$

Where μ_c is the mean of all points in cluster c :

$$\mu_c = \frac{1}{N_c} \sum_{j \in C_c} \phi(x_j)$$

Expanding the squared distance leads to the expression used in the code: $D^2(x_i, \mu_c) = K(i, i) - \frac{2}{N_c} \sum_{j \in C_c} K(i, j) + \frac{1}{N_c^2} \sum_{j \in C_c} \sum_{l \in C_c} K(j, l)$

- `K_diag` (Term 1: $K(i, i)$): The squared norm of the point $\phi(x_i)$.
- `term2` (Term 2: $-\frac{2}{N_c} \sum_{j \in C_c} K(i, j)$): The negative two times the dot product between the point $\phi(x_i)$ and the center μ_c .

- term3 (Term 3: $\frac{1}{N_c^2} \sum_{j \in C_c} \sum_{l \in C_c} K(j, l)$): The squared norm of the cluster center μ_c .

The algorithm iteratively updates the cluster assignments (`new_labels = np.argmin(dist_matrix, axis=1)`) until the assignments converge (stop changing) or the maximum iterations are reached.

spectral clustering

```
def spectral_clustering(K, k, h, w, mode, output_dir, init_method='random'):
    if not os.path.exists(output_dir):
        os.makedirs(output_dir)

    n = K.shape[0]
    W = K
    D_vec = np.sum(W, axis=1)
    D_vec[D_vec == 0] = 1e-10
    D = np.diag(D_vec)
    L = D - W

    if mode == 'ratio':
        eigvals, eigvecs = np.linalg.eigh(L)
        U = eigvecs[:, 1:k+1]

    elif mode == 'normalized':
        D_sqrt_inv = np.diag(1.0 / np.sqrt(D_vec))
        L_sym = np.dot(np.dot(D_sqrt_inv, L), D_sqrt_inv)

        eigvals, eigvecs = np.linalg.eigh(L_sym)
        U = eigvecs[:, :k]

        # Row normalization
        row_sums = np.sqrt(np.sum(U**2, axis=1)).reshape(-1, 1)
        row_sums[row_sums == 0] = 1e-10
        U = U / row_sums

    # Pass init_method to simple_kmeans
    labels = simple_kmeans(U, k, h, w, output_dir, init_method=init_method)
    plot_eigenspace(U, labels, mode, output_dir, k)

    return labels
```

The `spectral_clustering` function implements the core steps of the Spectral Clustering algorithm, which transforms the clustering problem from the original data space into an embedding space defined by the eigenvectors of a matrix derived from the similarity kernel K .

Affinity and Laplacian Matrices: The input K acts as the Affinity Matrix W .

Degree Matrix (D): A diagonal matrix where D_{ii} is the sum of the i -th row of W .

$$D_{ii} = \sum_j W_{ij}$$

Unnormalized Laplacian (L):

$$L = D - W$$

Clustering Modes: The code supports two common spectral clustering methods: Ratio Cut

- (Unnormalized): Clusters are found by finding the k smallest eigenvectors of the unnormalized Laplacian L (excluding the trivial first eigenvector).
- (Normalized): Clusters are found by finding the k smallest eigenvectors of the Symmetric Normalized Laplacian L_{sym} :

$$L_{sym} = D^{-1/2} L D^{-1/2}$$

- Embedding Matrix (U): The selected k eigenvectors form the embedding matrix U . Each row of U is the new feature vector for the corresponding pixel, mapping the data into a k -dimensional space where the clusters are expected to be linearly separable.

Final Step (Row Normalization & k -means): The rows of U are typically normalized to unit length. Standard k -means (simple_kmeans) is then applied to the rows of the normalized embedding matrix U to obtain the final pixel cluster assignments.

simple k means

```
def simple_kmeans(U, k, h, w, output_dir, max_iters=100, init_method='random'):
    """
    Standard K-means applied on Eigenvectors U
    """
    n, dim = U.shape

    # --- Modified: Use initialize_centers ---
    centers = initialize_centers(U, k, method=init_method, for_kernel=False)

    prev_labels = np.zeros(n)

    for it in range(max_iters):
        dists = np.sum((U[:, None, :] - centers[None, :, :]) ** 2, axis=2)
        labels = np.argmin(dists, axis=1)

        # Print less frequently to avoid clutter
        if it % 10 == 0: print(f" Standard K-means iter {it}")
        visualize_clusters(labels, h, w, f"{output_dir}/step_{it:03d}.png")

        if np.all(labels == prev_labels):
            break
        prev_labels = labels.copy()

        for i in range(k):
            mask = (labels == i)
            if np.any(mask):
                centers[i] = U[mask].mean(axis=0)
            else:
                centers[i] = U[np.random.randint(n)]

    return labels
```

The `simple_kmeans` function implements the standard k -means clustering algorithm, which partitions n data points into k clusters based on minimizing the variance within clusters.

1. Input: The data matrix U (where each row is a data point) and the number of clusters k .
2. Initialization: The algorithm starts by selecting k initial cluster centers (e.g., using 'random' or ' k -means++' methods).
3. Iterative Process: k -means alternates between two main steps:
 Assignment (E-step): Each data point x_i (row of U) is assigned to the nearest cluster center μ_j (row of centers). Distance is measured using Squared Euclidean Distance:

$$D^2(x_i, \mu_j) = \|x_i - \mu_j\|^2$$

Update (M-step): The cluster center μ_j is recalculated as the mean of all points C_j assigned to that cluster:

$$\mu_j = \frac{1}{|C_j|} \sum_{x_i \in C_j} x_i$$

4. Convergence: The process repeats until the cluster assignments no longer change (`np.all(labels == prev_labels)`).

2. Part2

```
if __name__ == "__main__":
    IMAGE_FILE = 'image2.png' # Ensure this file exists
    GAMMA_C = 1
    GAMMA_S = 1
    K_CLUSTERS = 2

    if not os.path.exists(IMAGE_FILE):
        print(f"Error: {IMAGE_FILE} not found. Please place an image file.")
    else:
        # Load and Precompute Gram Matrix once
        data, colors, coords, h, w = load_image(IMAGE_FILE)
        Gram_K = compute_gram_matrix(colors, coords, GAMMA_C, GAMMA_S)

        # Define experiments
        init_methods = ['random', 'kmeans++', 'fps', 'random_partition']
        base_output_dir = f"experiments/{IMAGE_FILE}/K_CLUSTERS={K_CLUSTERS}/GAMMA_C={GAMMA_C}_GAMMA_S={GAMMA_S}"

        if os.path.exists(base_output_dir):
            shutil.rmtree(base_output_dir) # Clean previous results

        print(f"\nStarting 4 x 3 Experiments...")
        print("="*40)
```

```
for method in init_methods:
    print(f"\n>>> Testing Initialization: {method.upper()}")

    # 1. Kernel K-means
    print(f" [1/3] Kernel K-means ({method})")
    dir_kkm = f"{base_output_dir}/{method}/kkm"
    kernel_k_means(Gram_K, K_CLUSTERS, h, w, output_dir=dir_kkm, init_method=method)
    make_gif(dir_kkm, f"{base_output_dir}/{method}_kkm.gif")

    # 2. Spectral (Ratio)
    print(f" [2/3] Spectral Ratio Cut ({method})")
    dir_ratio = f"{base_output_dir}/{method}/ratio"
    spectral_clustering(Gram_K, K_CLUSTERS, h, w, mode='ratio', output_dir=dir_ratio, init_method=method)
    make_gif(dir_ratio, f"{base_output_dir}/{method}_ratio.gif")

    # 3. Spectral (Normalized)
    print(f" [3/3] Spectral Normalized Cut ({method})")
    dir_norm = f"{base_output_dir}/{method}/norm"
    spectral_clustering(Gram_K, K_CLUSTERS, h, w, mode='normalized', output_dir=dir_norm, init_method=method)
    make_gif(dir_norm, f"{base_output_dir}/{method}_norm.gif")

print("\nAll experiments completed.")
```

The provided code segment is the main execution block of the clustering experiment. Its purpose is to load the image data, precompute the Gram Matrix K , and then iterate through different initialization methods to run and compare three different clustering algorithms.

3. Part3

```
def initialize_centers(data, k, method='random', for_kernel=False, K_matrix=None):
    n = data.shape[0] if data is not None else K_matrix.shape[0]

    # --- Logic for Kernel K-means (Return Labels) ---
    if for_kernel:
        if K_matrix is None:
            raise ValueError("K_matrix must be provided when for_kernel=True")

        K_diag = np.diag(K_matrix)

        # 1. Random Partition: 直接隨機給標籤
        if method == 'random_partition':
            return np.random.randint(0, k, n)

        # For 'random', 'kmeans++', 'fps': We first pick k "seed indices"
        centers_indices = []

        # Step A: Pick Seed Points
        if method == 'random':
            centers_indices = np.random.choice(n, k, replace=False)

        elif method in ['kmeans++', 'fps']:
            first_idx = np.random.randint(n)
            centers_indices.append(first_idx)

            for _ in range(k - 1):
                min_dists = np.full(n, np.inf)
                for c_idx in centers_indices:
                    # Kernel Distance:  $d^2(x, c) = K(x, x) + K(c, c) - 2K(x, c)$ 
                    dists_to_c = K_diag + K_diag[c_idx] - 2 * K_matrix[:, c_idx]
                    min_dists = np.minimum(min_dists, dists_to_c)

                min_dists = np.maximum(min_dists, 0) # Fix numerical error

            if method == 'kmeans++':
                if min_dists.sum() == 0:
                    probs = np.ones(n) / n
                else:
                    probs = min_dists / min_dists.sum()
                    cumprobs = np.cumsum(probs)
                    r = np.random.rand()
                    next_idx = np.searchsorted(cumprobs, r)
                    centers_indices.append(next_idx)

            elif method == 'fps':
                next_idx = np.argmax(min_dists)
                centers_indices.append(next_idx)

        # Step B: Convert Seeds to Labels
        dist_matrix = np.zeros((n, k))
        for i, c_idx in enumerate(centers_indices):
            dist_matrix[:, i] = K_diag + K_diag[c_idx] - 2 * K_matrix[:, c_idx]

        initial_labels = np.argmin(dist_matrix, axis=1)
        return initial_labels
```

```
# --- Logic for Standard K-means (Return Coordinates) ---
else:
    n, dim = data.shape

    if method == 'random':
        indices = np.random.choice(n, k, replace=False)
        return data[indices]

    elif method == 'kmeans++':
        centers = []
        first_center_idx = np.random.randint(n)
        centers.append(data[first_center_idx])

        for _ in range(k - 1):
            current_centers = np.array(centers)
            dists = np.sum((data[:, None, :] - current_centers[None, :, :])**2, axis=2)
            min_dists = np.min(dists, axis=1)

            if min_dists.sum() == 0:
                probs = np.ones(n) / n
            else:
                probs = min_dists / min_dists.sum()
            cumprobs = np.cumsum(probs)
            r = np.random.rand()
            ind = np.searchsorted(cumprobs, r)
            centers.append(data[ind])
        return np.array(centers)
```

```
elif method == 'fps':
    centers = []
    first_center_idx = np.random.randint(n)
    centers.append(data[first_center_idx])

    for _ in range(k - 1):
        current_centers = np.array(centers)
        dists = np.sum((data[:, None, :] - current_centers[None, :, :])**2, axis=2)
        min_dists = np.min(dists, axis=1)
        next_center_idx = np.argmax(min_dists)
        centers.append(data[next_center_idx])
    return np.array(centers)

elif method == 'random_partition':
    random_labels = np.random.randint(0, k, n)
    centers = np.zeros((k, dim))
    for i in range(k):
        mask = (random_labels == i)
        if np.any(mask):
            centers[i] = data[mask].mean(axis=0)
        else:
            centers[i] = data[np.random.randint(n)]
    return centers

else:
    raise ValueError(f"Unknown method: {method}")
```

The performance and final result of k -means (both standard and kernel) heavily depend on the choice of initial cluster centers. The function `initialize_centers` provides four methods to set these starting points.

1. random

- Logic: This is the simplest method. It randomly selects k data points (or indices, for the kernel version) directly from the dataset to serve as the initial cluster centers.
- Drawback: Because the selection is completely random, the initial centers may be clustered closely together, often leading to slow

convergence or suboptimal results (a poor local optimum).

2. kmeans++

- Logic: This method aims to select initial centers that are well-separated. The first center is chosen randomly. Subsequent centers are selected with a probability proportional to their squared distance from the nearest existing center.
- Goal: By favoring points that are far away from any already-chosen center, it ensures the initial centers are dispersed across the data space. This usually leads to faster convergence and better overall clustering quality.
- Metric: Uses the Squared Euclidean Distance for standard k -means and the Kernel Distance ($D^2(x, \mu)$) for Kernel k -means.

3. fps (Farthest Point Sampling)

- Logic: This method is similar to k -means++ but is deterministic. After the first center is chosen randomly, every subsequent center is selected as the point that is maximally distant from all currently chosen centers.
- Goal: This method rigorously maximizes the minimum separation between the initial cluster centers.

4. random_partition

- Logic: Instead of selecting k individual points as centers, this method first randomly assigns a label (1 to k) to every data point. It then calculates the mean of all points belonging to that randomly created partition, and this mean becomes the initial cluster center.
- Result: The initial centers are positioned closer to the center of mass for a random grouping of data.

4. Part4

```
def plot_eigenspace(U, labels, mode, output_dir, k):
    """
    Plots data points in the eigenspace (U), colored by their final cluster assignments (labels).
    This is used to visualize the separation of clusters in the feature space.
    """
    if k < 2:
        print("k is too small to plot 2D eigenspace.")
        return

    X_coord = U[:, 0]
    Y_coord = U[:, 1]

    # Define a color palette for visualizing clusters
    cluster_colors = np.array([
        [1, 0, 0], [0, 1, 0], [0, 0, 1],
        [1, 1, 0], [1, 0, 1], [0, 1, 1]
    ])

    # Map cluster labels to colors
    colors = cluster_colors[labels % len(cluster_colors)]

    # --- Plotting ---
    plt.figure(figsize=(8, 8))
    plt.scatter(X_coord, Y_coord, c=colors, s=5, alpha=0.5)

    plt.title(f"Eigenspace Scatter Plot ({mode} Cut, k={k})")
    plt.xlabel(f"Eigenvector 1 (U[:, 0])")
    plt.ylabel(f"Eigenvector 2 (U[:, 1])")

    # Save the figure
    filename = f"{output_dir}/eigenspace_plot_{mode}_cluster{k}.png"
    plt.savefig(filename)
    plt.close()
    print(f"Eigenspace plot saved to {filename}")
```

Its purpose is to visualize the cluster separation after the data has been transformed into the low-dimensional embedding space by the eigenvectors of the Laplacian matrix.

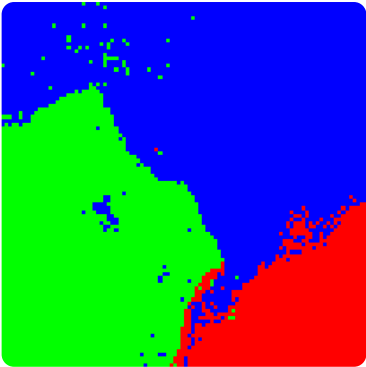
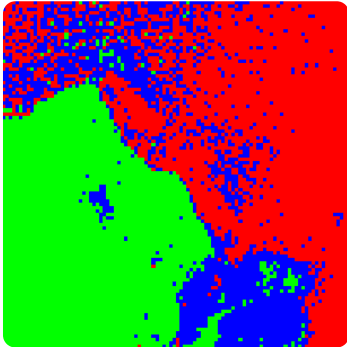
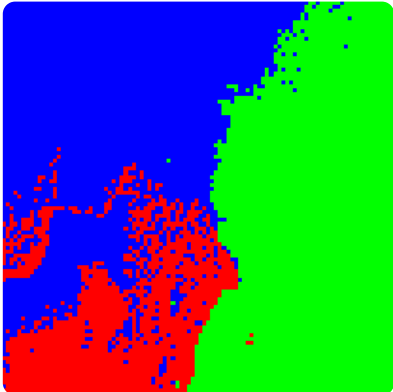
- Input: It takes the Embedding Matrix (U) (containing the selected eigenvectors) and the final cluster labels.
- Plotting: It creates a 2D scatter plot by using the first two eigenvectors (Eigenvector 1 and Eigenvector 2) as the X and Y coordinates.
- Visualization: Data points (pixels) are colored according to their final cluster assignment, clearly showing how the data is grouped in this new feature space, where the clusters are expected to be linearly separable for the subsequent k -means step.
- Constraint: It only proceeds if $k \geq 2$, as two dimensions are required for the scatter plot.

2. Experiments settings and results & discussion

i. Part1

hyperparameters setting: $k = 3$, $\gamma_c = 1$, $\gamma_s = 1$, init-method = random

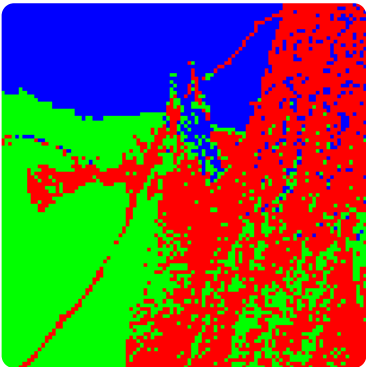
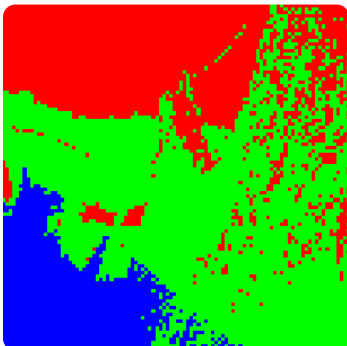
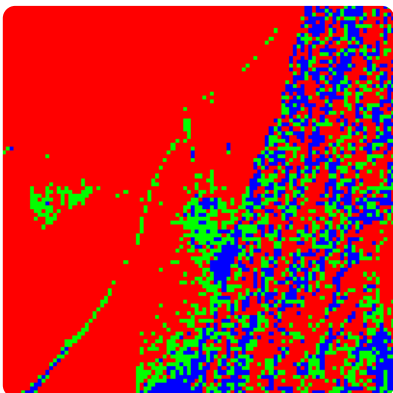
- image1

kernel k means	ratio cut	normalize cut
		

The filename of each images are

| image1_cluster3_random_kkm.gif | image1_cluster3_random_ratio.gif |
image1_cluster3_random_norm.gif |

- image2

kernel k means	ratio cut	normalize cut
		

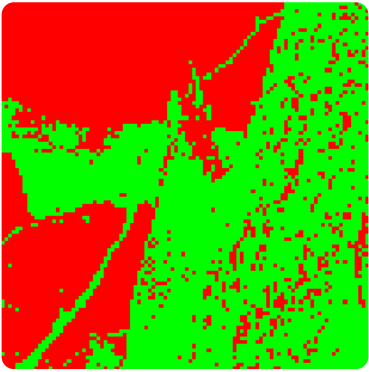
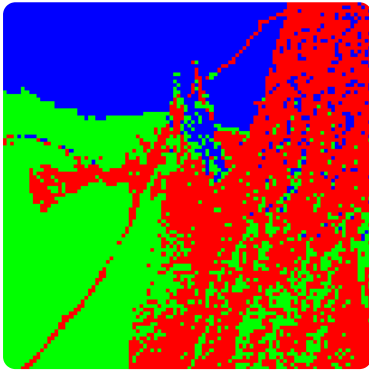
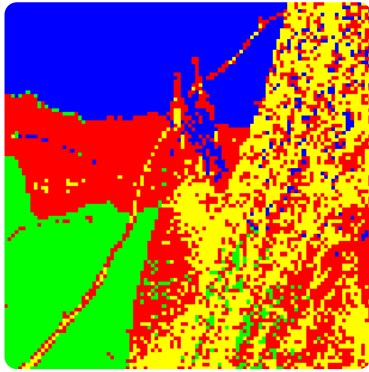
The filename of each images are

| image2_cluster3_random_kkm.gif | image2_cluster3_random_ratio.gif |
image2_cluster3_random_norm.gif |

ii. Part2

hyperparameters setting: $\gamma_c = 1$, $\gamma_s = 1$. Init-mehtod = random, Kernel k-means is the clustering algorithm

- image2

2 clusters	3 cluster3	4 clusters
		

The filename of each images are

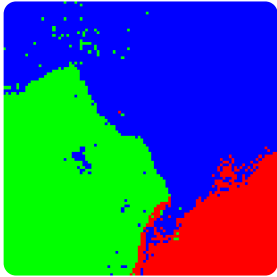
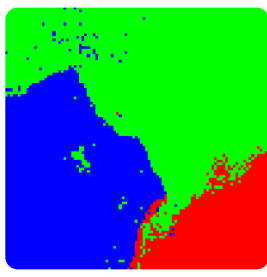
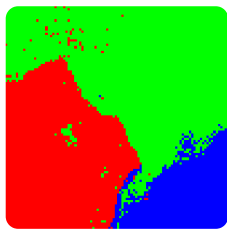
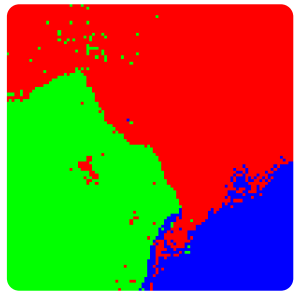
| image2_cluster2_random_kkm.gif | image2_cluster3_random_kkm.gif |
image2_cluster4_random_kkm.gif |

the number of clusrter is the hightly sensitive hyperparamerter and the tuning process relies on human. I think image2 needs 4 clusters.

iii. Part3

hyperparameters setting: $k = 3$, $\gamma_c = 1$, $\gamma_s = 1$. Kernel k-means is the clustering algorithm

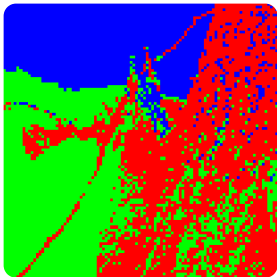
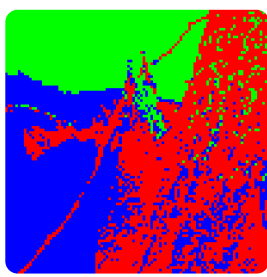
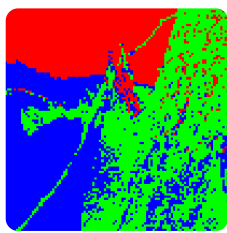
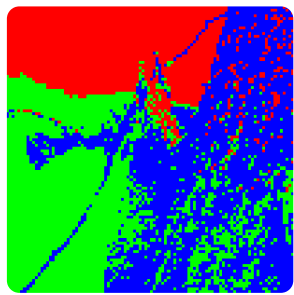
- image1

random	k-means ++	fps	random partition
			

The filename of each images are

| image1_cluster3_random_kkm.gif | image1_cluster3_kmeans++_kkm.gif |
 image1_cluster3_fps_kkm.gif | image1_cluster3_random_partition_kkm.gif

- image2

random	k-means ++	fps	random partition
			

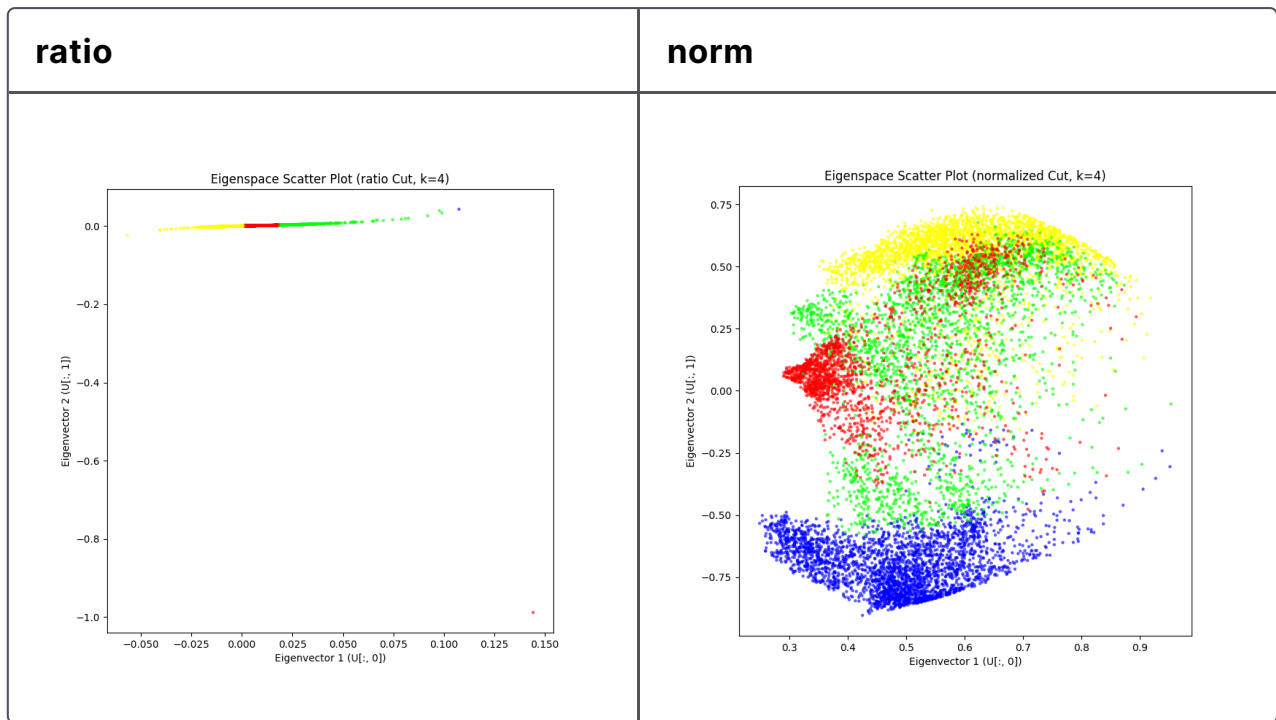
The filename of each images are

| image2_cluster3_random_kkm.gif | image2_cluster3_kmeans++_kkm.gif |
 image2_cluster3_fps_kkm.gif | image2_cluster3_random_partition_kkm.gif

iv. Part4

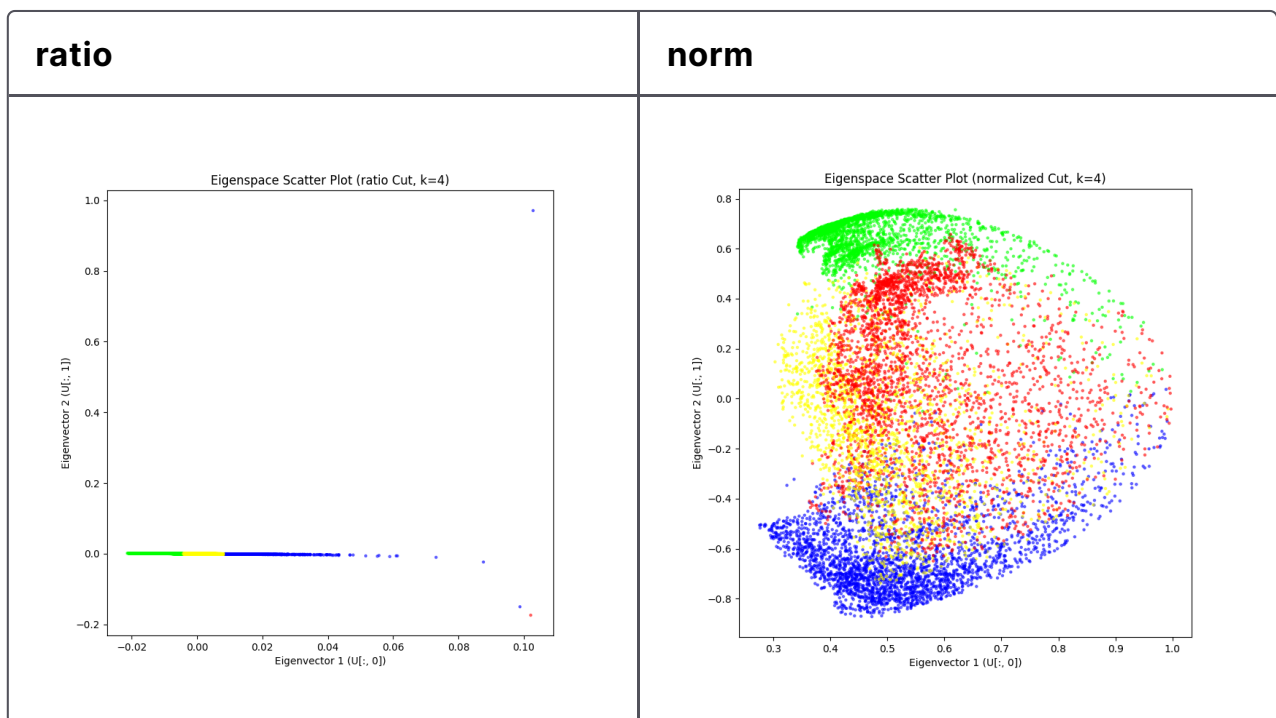
hyperparameters setting: $k = 3$, $\gamma_c = 1$, $\gamma_s = 1$. Kernel k-means is the clustering algorithm, init-method = random

- image1



The filename of each images are |
image1_cluster4_eigenspace_plot_ratio |
image1_cluster4_eigenspace_plot_normalized |

- image2



The filename of each images are |
 image2_cluster4_eigenspace_plot_ratio |
 image2_cluster4_eigenspace_plot_normalized |

3. Observations and discussion

i. Compare the performance between different clustering methods.

1. **Performance Disparity:** Ratio Cut vs. Normalized CutOur results demonstrate that Spectral Clustering with Ratio Cut performed significantly worse than Normalized Cut, often producing scattered or noisy boundaries. This behavior is theoretically expected due to the "Small Component Bias" inherent in the Ratio Cut objective function:

$$\text{RatioCut}(\{C_i\}) = \sum_{i=1}^k \frac{\text{Cut}(C_i, \bar{C}_i)}{|C_i|}$$

The unnormalized Laplacian ($L = D - W$) minimizes the cut relative to the number of nodes $|C_i|$. This penalizes large clusters and encourages the algorithm to isolate very small groups of pixels (small components) just to satisfy the minimization, failing to identify large, continuous image regions.

In contrast, Normalized Cut uses the Symmetric Normalized Laplacian (L_{sym}) and normalizes by the total degree volume ($\text{vol}(C_i)$). This formulation is less sensitive to cluster size, preventing the isolation of small artifacts and resulting in the cleaner, more balanced segmentation observed in our results.

2. **Kernel k -means vs. Spectral Clustering:**

While both methods rely on the same Gram Matrix K , their success relies on different mechanisms:

- Kernel k -means (Direct Clustering): This method minimizes within-cluster variance directly in the high-dimensional feature space Φ defined by the kernel. Its strong performance in this experiment suggests that our combined kernel (Color + Spatial) provided a high-quality similarity measure that was well-suited for direct optimization.
- Spectral Clustering (Projection + Clustering): This method operates as a two-step process: first using eigenvectors for dimensionality reduction to make clusters linearly separable in a low-dimensional embedding (U), and then applying simple k -means.

ii. Compare the execution time of different settings.

- hyperparameters setting: $k = 3$, $\gamma_c = 1$, $\gamma_s = 1$, image1

image1	Kernel K-means	Spectral Ratio	Spectral Normalized
Random	54.3381	358.4774	146.9593
Random_Partition	93.1806	134.2759	146.3883
Kmeans++**	78.0498	117.6098	162.3215
Fps	48.3762	121.8276	156.3722

- hyperparameters setting: $k = 3$, $\gamma_c = 0.5$, $\gamma_s = 0.5$, image2

Initialization Method	Kernel K-means	Spectral Ratio	Spectral Normalized
Random	84.9208	259.2186	226.1381
Kmeans++	53.3017	179.0293	182.2903
Fps	25.1905	157.9336	179.4662
Random_Partition	41.6354	134.8544	158.5916

1. Random Initialization: In the Spectral Ratio Cut algorithm, Random initialization took an extremely long time: 358.4774 seconds. This significantly exceeds all other initialization methods (e.g., Kmeans++ at 117.6098 seconds, FPS at 121.8276 seconds).
 - In Kernel K-means, Random initialization took 54.3381 seconds, which is competitive, but not the fastest (FPS was 48.3762 seconds).
2. Kmeans++ and FPS: These two methods show relatively stable and consistent computation times across all three algorithms.
 - In both Spectral Cut algorithms, the runtimes for Kmeans++ (117.61s) and FPS (121.83s) are close to each other and markedly lower than the extreme outlier produced by Random initialization for Spectral Ratio Cut.
 - FPS was the fastest overall for Kernel K-means (48.3762 seconds).

Discussion

The high runtime and instability of Random initialization, particularly in the Spectral Ratio Cut experiment, can be attributed to the following factors:

1. Susceptibility to Suboptimal Local Minima:
 - The performance of the K-means algorithm (which is the final step in Spectral Clustering) is highly dependent on the choice of initial cluster centers. Random initialization selects initial centers haphazardly, often leading to poor starting configurations where initial centers are too close or not representative of the cluster centers.
 - This poor start forces the K-means algorithm to take significantly more iterations to converge, if it converges well at all, resulting in longer execution times.

- From the logs, we can compare the iterations for Kernel K-means (KKM):
 - KKM (random): 12 iterations (54.34s)
 - KKM (kmeans++): 8 iterations (78.05s)
 - KKM (fps): 11 iterations (48.38s)
 - KKM (random_partition): 27 iterations (93.18s)
- While KKM (random) had fewer iterations than KKM (random_partition), the extreme outlier in Spectral Ratio Cut (358.48s) strongly suggests that the Standard K-means step used on the eigenvectors was forced into a very slow or complex convergence path due to poor random initial centroids.

2. Spectral Clustering Sensitivity:

- Spectral Clustering involves transforming the data into a lower-dimensional embedding space (the Eigenspace) where K-means is applied. If the initial random centroids in this Eigenspace are poorly chosen, the K-means clustering is highly inefficient.

3. The Advantage of K-means++ and FPS (Stability):

- K-means++ and Farthest Point Sampling (FPS) are "smart initialization" methods designed to overcome the flaws of simple random initialization. They strategically select initial centers that are far apart and well-distributed.
- K-means++ selects new centers with a probability proportional to the squared distance from existing centers.

- FPS selects the point farthest from all existing centers.
- This higher quality initialization allows the K-means algorithm to converge much faster, often in fewer iterations, leading to the stable and predictable runtimes observed across all three clustering methods, especially preventing the extreme time penalty seen with Random initialization in the Spectral Ratio Cut case.

iii. Different initialization

1. Stability of Heuristic Methods (kmeans++ and fps)

Our experiments confirm that kmeans++ and fps (Farthest Point Sampling) provide the most consistent and reliable segmentation results.

- Mechanism: Both methods explicitly design the initial centers to be well-separated. kmeans++ uses probability proportional to distance, while fps deterministically selects the farthest point.
- Result: This strategy effectively covers the data manifold from the start, preventing the "crowded center" problem where multiple centers compete for the same region. This leads to faster convergence and consistently high-quality segmentation.

2. Inconsistency of Random Methods (random and random_partition)

The random and random_partition methods showed high variance in performance.

- Mechanism: random selects points arbitrarily, and random_partition calculates means of arbitrary groups.
- Risk: These methods blindly ignore the data structure. There is a significant risk that initial centers will be clumped together or placed in outliers. While they can produce good results (as seen in some of our successful runs), they are prone to getting stuck in poor local optima, leading to suboptimal segmentation

3. Robustness of Kernel k -means

A key observation from the results is that Kernel k -means demonstrated a degree of resilience to bad initialization.

- Mitigation: While standard k -means is theoretically sensitive to initialization, the Kernel k -means algorithm in this experiment uses a highly discriminative Combined Kernel ($K_s \odot K_c$).
- Explanation: This kernel creates a strong gradient in the feature space that separates pixels effectively based on color and position. This strong "signal" can sometimes guide the algorithm toward the correct global structure, allowing it to recover from a sub-optimal random start better than spectral methods, which rely heavily on the quality of the initial eigenvector embedding.